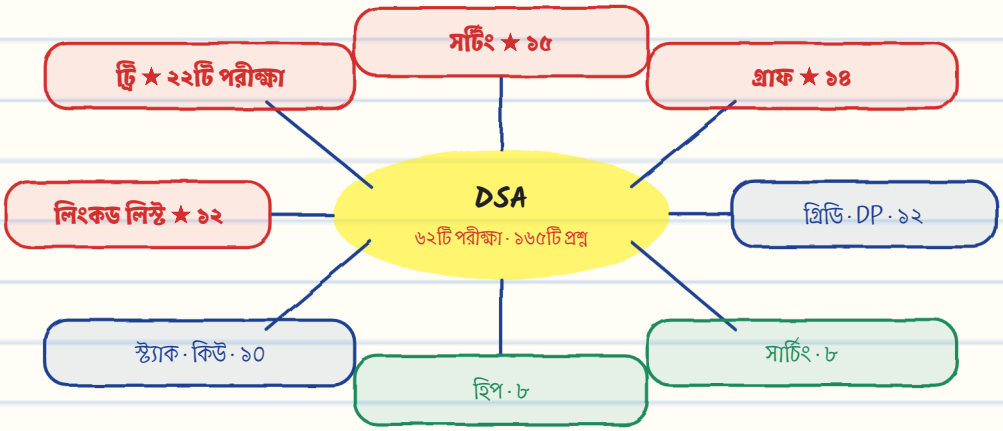
ডেটা স্ট্রাকচার ও অ্যালগরিদম

BCS · NTRCA · BPSC · ব্যাংক · BUET-এর প্রশ্নে আসা সব DSA প্রশ্ন, ২৩ পাতায় সমাধান



↑ সংখ্যাটি হলো কতগুলো আলাদা পরীক্ষায় টপিকটি এসেছে (লাল = সবচেয়ে বেশি)

➜ প্রশ্ন-কার্ড যেভাবে পড়বেন

প্র প্রশ্নটি, প্রশ্নপত্রে যেমন ছাপা (বানান ঠিক করা)।
✎ **যে পরীক্ষায় এসেছে:** যে পরীক্ষাগুলো থেকে নেওয়া, যেমন BPSC AP 2023 · 45th BCS CSE 2024

তারিখসহ পরীক্ষা না থাকলে কার্ডে উৎস লেখা থাকে (NTRCA প্রশ্নব্যাংক, ব্যাংক আইটি ভাইভা)। সবুজ = সমাধান / আউটপুট।

সবচেয়ে বেশি আসে
① অ্যারে বনাম লিংকড লিস্ট (৫)
② ট্রাভার্সাল → ট্রি পুনর্গঠন
③ BFS বনাম DFS
④ কুইক / মার্জ সর্টের খরচ
⑤ স্ট্যাক বনাম কিউ
⑥ D&C বনাম DP
⑦ হিপ তৈরি

➜ সিলেবাসের যে অংশ

"Internal data representation; Abstract data types; Elementary data structures: arrays, lists, stacks, queues, trees, graphs; Advanced: heaps, Fibonacci heaps, B-trees; Recursion, sorting, searching, hashing; analysis of algorithms; design: divide and conquer, greedy, dynamic programming; graph algorithms; NP-completeness"।

সংকেত হলুদ = মূল শব্দ · লাল বৃত্ত = মুখস্থ রাখুন · **বেগুনি বাক্স** = মনে রাখার কৌশল · **লাল বাক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধান / আউটপুট

➜ সূচিপত্র

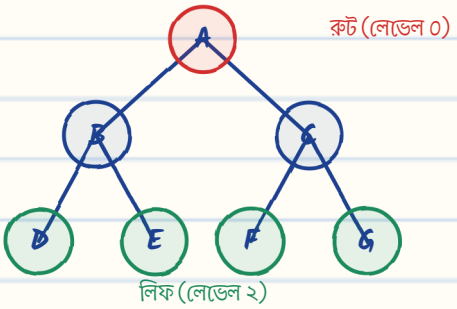
✓ ট্রি ১–৪ ..... পৃ. ২–৫
✓ সর্টিং ১–৩ ..... পৃ. ৬–৮
✓ গ্রাফ ১–৩ ..... পৃ. ৯–১১
✓ লিংকড লিস্ট ..... পৃ. ১১–১২
✓ স্ট্যাক ও কিউ ..... পৃ. ১৩–১৪
✓ হিপ; সার্চিং, হ্যাশিং ... পৃ. ১৪–১৬
✓ অ্যালগরিদম ডিজাইন ১–৩ ..... পৃ. ১৭–১৯
✓ নোটেশন, মৌলিক, অ্যারে ... পৃ. ২০–২১
✓ পরীক্ষার আগের রাত ..... পৃ. ২২
✓ অনুশীলন ..... পৃ. ২৩

১ বাইনারি ট্রি ও নোডের সংখ্যা

ট্রি এসেছে ২২টি পরীক্ষায়

প্র১ *Binary Tree* কী? উদাহরণসহ ব্যাখ্যা করুন এবং *Binary Tree Traversing*-এর পদ্ধতিসমূহ আলোচনা করুন।

✎ **যে পরীক্ষায় এসেছে:** *W2PGCL AE(CSE) 2023 · 18th NTRCA College ICT 2024*



বাইনারি ট্রি এমন ট্রি যার প্রতিটি নোডের **সর্বোচ্চ দুটি সন্তান** থাকে: বাম ও ডান। উদাহরণ: এক্সপ্রেশন ট্রি, BST, হিপ।

ট্রাভার্সাল	নিয়ম	এই ট্রিতে
Preorder	*Root, L, R*	*A B D E C F G*
Inorder	*L, Root, R*	*D B E A F C G*
Postorder	*L, R, Root*	*D E B F G C A*
Level order	লেভেল ধরে (কিউ)	*A B C D E F G*

প্র২ উচ্চতা ৭ (রুটের উচ্চতা ০) এমন বাইনারি ট্রিতে সর্বোচ্চ ও সর্বনিম্ন নোড কত?

✎ **যে পরীক্ষায় এসেছে:** *NESCO AM(ICT) 2021*

সর্বোচ্চ: প্রতিটি লেভেল পূর্ণ → $2^0 + \dots + 2^7 = 2^8 - 1$। সর্বনিম্ন: প্রতি লেভেলে একটি → $h + 1$।

→ সর্বোচ্চ **255**, সর্বনিম্ন **8**

প্র৩ *N* লেভেলের বাইনারি ট্রিতে সর্বোচ্চ কতটি নোড? বাইনারি ট্রিতে সর্বনিম্ন কতটি নোড?

✎ **যে পরীক্ষায় এসেছে:** *Combined Bank AP 2020 · BCC AP 2023*

লেভেল i-তে সর্বোচ্চ 2^{i-1}টি নোড (লেভেল ১ থেকে)।

→ $2^N - 1$ · সর্বনিম্ন **0** (খালি ট্রিও বাইনারি ট্রি); উচ্চতা h হলে $h + 1$

প্র৪ প্রমাণ করুন: *proper* বাইনারি ট্রিতে লিফ নোড ইন্টারনাল নোডের চেয়ে একটি বেশি।

✎ **যে পরীক্ষায় এসেছে:** *Titas Gas AE(CSE) 2024* (ছাপা "one more node is internal node")

Proper (full) বাইনারি ট্রিতে প্রতিটি ইন্টারনাল নোডের ঠিক ২টি সন্তান। ধরি i = ইন্টারনাল নোড, l = লিফ, $n = i + l$। রুট ছাড়া প্রতিটি নোডের একটি প্যারেন্ট-এজ: এজ $= n - 1$। প্রতিটি ইন্টারনাল নোড ২টি এজ দেয়: এজ $= 2i$। তাই $2i = i + l - 1 \rightarrow l = i + 1$। ওপরের ট্রিতে যাচাই: $i = 3$ (A, B, C), $l = 4$ ✓।

বাইনারি ট্রির ধরন	নিয়ম	মন্তব্য
Full (proper)	প্রতিটি নোডের ০ বা ২ সন্তান	লিফ = ইন্টারনাল + ১
Complete	শেষটি ছাড়া সব লেভেল পূর্ণ, শেষটি বাম থেকে ভরা	হিপ; অ্যারেতে মানায়
Perfect	সব ইন্টারনাল নোডের ২ সন্তান, সব লিফ এক লেভেলে	$2^{h+1} - 1$টি নোড
Skewed	প্রতিটি নোডের একটি সন্তান	লিস্টের মতো, উচ্চতা $n - 1$

প্র৫ অ্যারে ব্যবহার করে বাইনারি ট্রি কীভাবে উপস্থাপন করবেন?

✎ **যে পরীক্ষায় এসেছে:** BGDCL AM(CSE) 2024 · MRA SCO 2023

ইনডেক্স	1	2	3	4	5	6	7
নোড	A	B	C	D	E	F	G

মনে রাখার কৌশল i-তে থাকা নোডের: **বাম** $2i$, **ডান** $2i + 1$, প্যারেন্ট $\lfloor i/2 \rfloor$ (রুট 1-এ)। রুট 0-তে হলে: $2i + 1$, $2i + 2$।

লেভেল ধরে রাখা হয়; কোনো সন্তান না থাকলে একটি **খালি ঘর** থাকে, তাই h উচ্চতার একপাশে হেলে থাকা ট্রি $2^{h+1} - 1 - (h + 1)$টি ঘর নষ্ট করতে পারে: অ্যারে ***complete*** ট্রির (হিপ) জন্য উপযুক্ত; নইলে লিংকড নোড (*data, left, right*)।

২ ট্রাভার্সাল থেকে ট্রি পুনর্গঠন

BCS · BPDB · BUET

প্র৬ একটি বাইনারি ট্রির *in-order* ও *preorder* ট্রাভার্সাল যথাক্রমে *dbeafcg* ও *abdecfg*। *post-order* ট্রাভার্সাল কী হবে?

✎ **যে পরীক্ষায় এসেছে:** 45th BCS CSE 2024

১. *Preorder*-এর প্রথম অক্ষর রুট: **a**। *Inorder*-এ *a*-এর বামে *d b e*, ডানে *f c g*

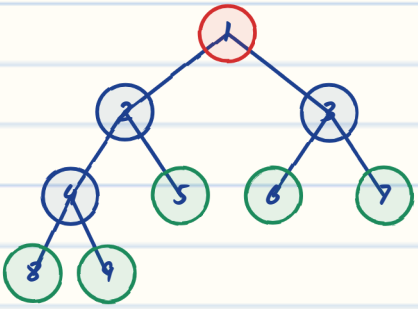
২. *Preorder*-এ পরেরটি **b** বাম অংশের রুট (*d | b | e*); তারপর **c** ডান অংশের রুট (*f | c | g*)

৩. ট্রি: *a(b(d, e), c(f, g))*; *Left-Right-Root* ক্রমে পড়ুন

→ *post-order* **d e b f g c a**

প্র৭ *pre[] = {1, 2, 4, 8, 9, 5, 3, 6, 7}*, *post[] = {8, 9, 4, 5, 2, 6, 7, 3, 1}*: বাইনারি ট্রিটি আঁকুন।

✎ **যে পরীক্ষায় এসেছে:** BPDB AE(CSE) 2024 · BUET M.Sc. April 2016



১. রুট = *pre[0]* = 1 = *post*-এর শেষটি

২. *pre[1]* = 2 বাম সন্তান; *post*-এ 2 পর্যন্ত সব (8 9 4 5 2) বাম সাবট্রি, বাকি (6 7 3) ডান

৩. আবার: 2 → (4 → 8, 9), 5; 3 → 6, 7

৪. *Inorder* যাচাই: 8 4 9 2 5 1 6 3 7 ✓

পরীক্ষার ফাঁদ *Pre + post* ট্রিকে নির্দিষ্ট করে **শুধু *full* হলে** (প্রতিটি নোডের ০ বা ২ সন্তান); একটি সন্তান থাকলে বাম না ডান বোঝা যায় না। *Pre + in* বা *post + in* সবসময় কাজ করে। এক লাইনে এটা লিখে দিন।

প্র৮ Preorder 1 5 8 7 4 9 3 ও inorder 8 5 7 1 9 4 3 থেকে ট্রি তৈরি করুন।
✎ **যে পরীক্ষায় এসেছে:** APSCL AE(ICT) 2021 (ছাপা "construct a BST")

রুট 1; বাম (in 8 5 7, pre 5 8 7) → 5(8, 7); ডান (in 9 4 3, pre 4 9 3) → 4(9, 3)। ট্রি **1(5(8, 7), 4(9, 3))**, postorder 8 7 5 9 3 4 1। এটি বাইনারি ট্রি, **BST নয়** (8 > 5 হয়েও 5-এর বামে): তা লিখে দিন।

প্র৯ a ... i নোডবিশিষ্ট একটি বাইনারি ট্রির post-order a b f c h d e g i। ট্রিটি আঁকুন এবং এর sequential (অ্যারে) রূপ দিন।
✎ **যে পরীক্ষায় এসেছে:** Petrobangla AM(CSE) 2024 (তিনটি উৎস)

সাধারণভাবে একক নয়; অ্যারে রূপ মানে ৯ নোডের **complete** ট্রি। তার postorder অবস্থান 8 9 4 5 2 6 7 3 1, তাই a b f c h d e g i দিয়ে ভরুন: রুট i; i → (h, g); h → (f, c); g → (d, e); f → (a, b)।

1	2	3	4	5	6	7	8	9
i	h	g	f	c	d	e	a	b

ডান, বাম, রুট ✎ **যে পরীক্ষায় এসেছে:** DPDC AM(ICT) 2025 (ট্রি ছাপা নেই): preorder উল্টো দিক থেকে পড়লে এটি পাওয়া যায়; পৃ. ২-এর ট্রিতে → G F C E D B A।

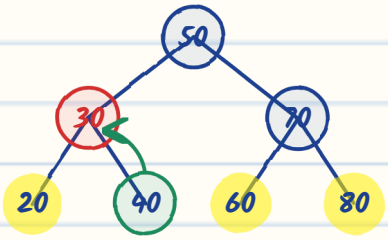
· **১৫ নোডের ট্রির ট্রাভার্সাল** ✎ **যে পরীক্ষায় এসেছে:** BPSC AP 2023: উচ্চতা ৩-এর একটি full ট্রি আঁকুন (1 ... 15 লেভেল ধরে) এবং পৃ. ২-এর টেবিল প্রয়োগ করুন।

ট্রির BFS ও DFS 1 → (2, 3), 2 → (4, 5) ✎ **যে পরীক্ষায় এসেছে:** BAPEX AGM(ICT) 2023: BFS 1 2 3 4 5 · DFS (preorder) 1 2 4 5 3 · inorder 4 2 5 1 3 · postorder 4 5 2 3 1। · Pre + in ধাপে ধাপে ✎ **যে পরীক্ষায় এসেছে:** BSCL AE(CSE) 2026: প্র৬-এর পদ্ধতি।

৩ বাইনারি সার্চ ট্রির অপারেশন

NTRCA · ব্যাংক · PGCB

প্র১০ Binary Search Tree-এর অপারেশন ব্যাখ্যা করুন। BST-তে ডেটা যোগ করুন, post-order-এ প্রিন্ট করুন, একটি নোড মুছে বৈধ BST আবার আঁকুন।
✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE 2023 · Combined Bank SO(IT) 2020 · যোগ, মোছা, আবার আঁকা: PGCB AE(CSE) 2024 (ডেটা ছাপা নেই)



30 মুছলে → সাকসেসর 40 ওপরে ওঠে

- ✓ **Search**: রুটের সাথে তুলনা, ছোট হলে বামে, বড় হলে ডানে; O(h)
- ✓ **Insert** 50, 30, 70, 20, 40, 60, 80: খুঁজে যেখানে পথ শেষ সেখানে বসান
- ✓ Post-order: 20 40 30 60 80 70 50
- ✓ **Delete**: লিফ → সরান; এক সন্তান → সন্তানকে ওপরে যুক্ত করুন; দুই সন্তান → inorder successor (ডান সাবট্রির সবচেয়ে ছোট) দিয়ে প্রতিস্থাপন
- ✓ 30 মুছলে → 50(40(20), 70(60, 80)); post-order 20 40 60 80 70 50

প্র১১ বাইনারি সার্চ ট্রিতে একটি নোড খোঁজার অ্যালগরিদম লিখুন।
✎ **যে পরীক্ষায় এসেছে:** PSB AP 2018

```
Node* search(Node *t, int key) {
    while (t != NULL && t->key != key)
        t = (key < t->key) ? t->left : t->right;
    return t;              /* NULL = not found; O(h): O(log n) balanced, O(n) skewed */
}
```

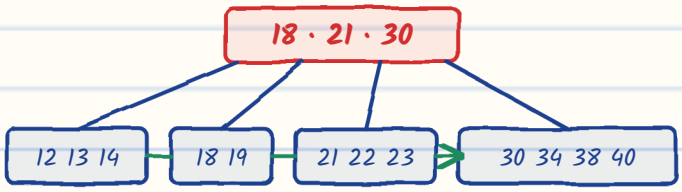
মনে রাখার কৌশল "**BST-র inorder সাজানো**": 20 30 40 50 60 70 80। তাই MCQ "কোন ট্রাভার্সাল BST সাজায়?" → **inorder** ✎ **যে পরীক্ষায় এসেছে:** 49th BCS ICT 2025।

৪ B-tree ও B+ tree NTRCA · BUET

প্র১২ B-tree data structure কী? এর প্রয়োগ ব্যাখ্যা করুন।
✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE 2023 · ইনডেক্সিংয়ে B+ tree: NTRCA প্রশ্নব্যাংক

m অর্ডারের B-tree একটি ব্যালান্সড বহুমুখী সার্চ ট্রি: প্রতিটি নোডে সর্বোচ্চ $m-1$টি সাজানো কী ও m সন্তান, রুট ছাড়া প্রতিটি নোড অন্তত অর্ধেক পূর্ণ, সব লিফ একই লেভেলে, উচ্চতা $O(\log n)$। কম কিন্তু চওড়া নোড = কম **ডিস্ক রিড**: তাই **ডেটাবেজ ও ফাইল সিস্টেমের ইনডেক্সে** ব্যবহৃত (MySQL InnoDB, NTFS)। **B+ tree** সব কী লিংকড লিফে রাখে, ইন্টারনাল নোডে শুধু কপি, তাই range scan দ্রুত।

প্র১৩ $n = 5$ নিয়ে কী 12, 14, 19, 30, 21, 22, 13, 34, 18, 23, 38, 40 দিয়ে একটি B+ tree আঁকুন।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. April 2017



$n = 5$ পয়েন্টার → প্রতি নোডে সর্বোচ্চ **৪টি কী**। ভরা লিফ 3 | 2 ভাগ হয় এবং ডান অংশের প্রথম কী ওপরে **কপি** হয়: 21 যোগে [12 14 19 21 30] ভাঙে → রুট 21; 18 যোগে [12 13 14 18 19] ভাঙে → 18 ওপরে; 23 যোগে [21 22 23 30 34] ভাঙে → 30 ওপরে। 38, 40 শেষ লিফ ভরায়।

Decision tree ✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE 2023: যে ট্রির ইন্টারনাল নোড পরীক্ষা ও লিফ ফলাফল (যেমন "আবহাওয়া? → আর্দ্রতা? → খেলা হবে")। তুলনা দিয়ে সর্টিং $n!$টি লিফের decision tree, উচ্চতা $\geq \log_2 n! = \Omega(n \log n)$: MCQ "সর্টিংয়ের নিম্নসীমা প্রমাণে ব্যবহৃত হয় ***decision tree***" ✎ **যে পরীক্ষায় এসেছে:** 49th BCS ICT 2025।

৫ এক টেবিলে দশ প্রশ্নের উত্তর

সর্টিং এসেছে ১৫টি পরীক্ষায়

প্র১৪ অ্যালগরিদমের *complexity* বলতে কী বোঝায়? প্রচলিত সর্টিং অ্যালগরিদমগুলোর *best, worst* ও *average time complexity* লিখুন।

✎ **যে পরীক্ষায় এসেছে:** *17th NTRCA CSE 2023 · DPDC AM(ICT) 2025 · bubble, quick, merge-এর worst case: BTCL JAM 2022 · merge, heap, insertion: BBL AM(ICT) 2019*

অ্যালগরিদম	Best	Average	Worst	Space	Stable	মন্তব্য
Bubble	n	n^2	n^2	1	হ্যাঁ	"swap না হলে থামো" ফ্ল্যাগে best
Selection	n^2	n^2	n^2	1	না	সবচেয়ে কম swap ($n-1$)
Insertion	**n**	n^2	n^2	1	হ্যাঁ	ছোট / প্রায় সাজানো ডেটায় সেরা
Merge	$n \log n$	$n \log n$	$n \log n$	n	হ্যাঁ	নিশ্চিত, বাড়তি অ্যারে লাগে
Quick	$n \log n$	$n \log n$	**n^2**	$\log n$	না	বাস্তবে সবচেয়ে দ্রুত
Heap	$n \log n$	$n \log n$	$n \log n$	1	না	in place ও নিশ্চিত
Counting / radix	$n + k$	$n + k$	$n + k$	$n + k$	হ্যাঁ	তুলনা-ভিত্তিক নয়

Complexity = ইনপুটের আকার n বাড়লে অ্যালগরিদমের চলার সময় (*time complexity*) বা মেমোরি (*space complexity*) কীভাবে বাড়ে, ঊর্ধ্বসীমা Big-O দিয়ে লেখা হয়। যেকোনো তুলনা-ভিত্তিক সর্ট *worst case*-এ $\Omega(n \log n)$ লাগে।

যে MCQ এসেছে *Insertion sort*-এর *worst-case* তুলনা → $n(n-1)/2$ ✎ **যে পরীক্ষায় এসেছে:** *49th BCS ICT 2025* · তুলনা-ভিত্তিক নয় → ***counting sort*** ✎ **যে পরীক্ষায় এসেছে:** *49th BCS ICT 2025* · $S = \{1, 2, 3, 8, 15, 10\}$ সাজাতে দ্রুততম → ***insertion sort***: প্রায় সাজানো, ৬টি তুলনা ✎ **যে পরীক্ষায় এসেছে:** *Combined Bank AP 2020*

পরীক্ষার ফাঁদ "সবচেয়ে দ্রুত সর্টিং?" ✎ **যে পরীক্ষায় এসেছে:** *BCC AP 2023*-এর একক উত্তর নেই: গড়ে বাস্তবে ***quick sort***, নিশ্চিত $n \log n$-এর জন্য ***merge / heap***, প্রায় সাজানো ডেটায় ***insertion***, ছোট পূর্ণসংখ্যা কী-তে ***counting / radix***। চারটি লাইনই লিখুন।

প্র১৫ *Bubble sort* অ্যালগরিদম লিখুন। এ অ্যালগরিদমটির *Time Complexity* বের করুন।

✎ **যে পরীক্ষায় এসেছে:** *BPSC AP 2021 · বড় ডেটায় bubble sort কেন merge sort-এর চেয়ে ধীর: ব্যাংক AP লিখিত (শিটে ব্যাংকের নাম কাটা)*

```
for (i = 0; i < n-1; i++) {
  swapped = 0;
  for (j = 0; j < n-1-i; j++)
    if (a[j] > a[j+1]) {
      swap(a[j], a[j+1]);
      swapped = 1;
    }
  if (!swapped) break;
}
```

5 8 3 6 2 →
পাস ১: 5 3 6 2 **8**
পাস ২: 3 5 2 **6 8**
পাস ৩: 3 2 **5 6 8**
পাস ৪: **2 3 5 6 8** (১০টি তুলনা)

তুলনা $(n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2}$ → $\mathbf{O(n^2)}$; আগে থেকে সাজানো → এক পাস, $O(n)$। 10^6টি উপাদানে $n^2 = 10^{12}$ ধাপ, merge sort-এ $n \log n \approx 2 \times 10^7$।

ছাপা প্রশ্ন "অসাজানো অ্যারেতে *bubble sort* কেন ব্যর্থ হয়?"-এর ভিত্তিই ভুল: *bubble sort* যেকোনো অ্যারেতে কাজ করে। অসাজানো ডেটায় ব্যর্থ হয় **binary search**।

৬ কুইক সর্ট

BCS · BPSC · Petrobangla

প্র১৬ রিকার্সিভ পদ্ধতিতে *quick sort*-এর *Pseudo code* লিখুন এবং এর *average complexity* লিখুন। *Quick sort* কীভাবে বাস্তবায়ন হয়?

✎ **যে পরীক্ষায় এসেছে:** *BPSC ADA 2022 · BPSC AME 2021 · quick sort কীভাবে কাজ করে: NTRCA registration viva 2023*

```
quickSort(A, lo, hi):
  if lo < hi:
    p = partition(A, lo, hi)
    quickSort(A, lo, p - 1)
    quickSort(A, p + 1, hi)
partition(A, lo, hi):      // Lomuto
  pivot = A[hi]; i = lo - 1
  for j = lo .. hi-1:
    if A[j] < pivot: i++; swap A[i], A[j]
  swap A[i+1], A[hi]; return i + 1
```

10 80 30 90 40 50 70, pivot 70 →
10 30 40 50 **70** 90 80 (70 চূড়ান্ত স্থানে)
সাজানো: 10 30 40 50 70 80 90

গড়: সমান ভাগ, $T(n) = 2T(n/2) + n$ → $O(n \log n)$; in place, $O(\log n)$ স্ট্যাক।

প্র১৭ *Quicksort*-এর *worst-case complexity* কেন $O(n^2)$? প্রথম উপাদান *pivot* এবং অ্যারে আগে থেকে সাজানো থাকলে বিশ্লেষণ করুন।

✎ **যে পরীক্ষায় এসেছে:** *45th BCS CSE 2024 · Petrobangla AM(CSE) 2024 · randomized quicksort-এর worst case (MCQ): Combined Bank Officer(IT) 2019*

সাজানো ইনপুট + প্রথম *pivot*: *pivot* সবচেয়ে ছোট, তাই প্রতিবার ভাগ হয় **0 | n − 1**। $T(n) = T(n-1) + (n-1) = (n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2 = O(n^2)}$ (গুনে দেখা: $n = 10$ → ৪৫টি তুলনা)। র‍্যান্ডম *pivot* এটাকে অসম্ভাব্য করে, কিন্তু **worst case তবুও $O(n^2)$**; প্রত্যাশিত $O(n \log n)$।

৭ মার্জ সর্ট *divide and conquer*

প্র১৮ *Divide and conquer* কৌশলে *merge sort* অ্যালগরিদম লিখুন। $T(n) = 2T(n/2) + n$ সমীকরণটি কীভাবে আসে ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** *45th BCS CSE 2024 · সমীকরণ: BPSC AP 2023*

```
mergeSort(A, l, r):
  if l < r:
    m = (l + r) / 2
    mergeSort(A, l, m)      // T(n/2)
    mergeSort(A, m+1, r)    // T(n/2)
    merge(A, l, m, r)       // n
```

ভাগ করা $O(1)$, **দুই অর্ধেক** $2T(n/2)$, **merge** প্রতিটি উপাদান একবার ছোঁয়: n। তাই $T(n) = 2T(n/2) + n$। *Recursion tree:* $\log_2 n$টি লেভেল × প্রতিটিতে n কাজ = $O(n \log n)$ *(master theorem case 2)*।

38 27 43 3 9 82 10 → 3 9 10 27 38 43 82

প্র১৯ দুটি সাজানো তালিকা মার্জ করার একটি *linear* অ্যালগরিদম লিখুন। এটি কেন $O(n)$?

✎ **যে পরীক্ষায় এসেছে:** *SGFL AE(IT) 2023*

```
i = j = k = 0
while i < n and j < m:
  Z[k++] = (X[i] <= Y[j]) ? X[i++] : Y[j++]
copy the rest of X, then the rest of Y
```

প্রতি ধাপে একটি উপাদান Z-এ যায়, কখনো পেছনে ফেরে না: সর্বোচ্চ **$n + m - 1$টি তুলনা** → $O(n + m)$। *X = 2 5 9, Y = 1 6 7 10 → 1 2 5 6 7 9 10 (৬টি তুলনা)*

কুইক বনাম মার্জ ✎ **যে পরীক্ষায় এসেছে:** *BPSC ME 2021*	*Quick sort*	*Merge sort*
সময়	গড়ে $n \log n$, worst n^2	সবসময় $n \log n$
মেমোরি	*in place*, $O(\log n)$ স্ট্যাক	$O(n)$ বাড়তি অ্যারে
মূল কাজ	*partition* (রিকার্শনের আগে)	*merge* (রিকার্শনের পরে)
Stable / উপযুক্ত	না / RAM-এর অ্যারে	হ্যাঁ / লিংকড লিস্ট, বাহ্যিক সর্টিং

Best ও worst case ✎ **যে পরীক্ষায় এসেছে:** *RPGCL AM(ICT) 2022: binary search $O(1)$ / $O(\log n)$ · quick sort $O(n \log n)$ / $O(n^2)$ (ছাপা best case $O(n)$ ভুল) · DFS দুটোতেই $O(V + E)$।*

প্র২০ সিলেকশন সর্ট পদ্ধতির অ্যালগরিদম লিখুন এবং 45, 72, 80, 65, 84, 52, 37-কে ধাপে ধাপে নিম্নক্রমে সাজান।

✎ **যে পরীক্ষায় এসেছে:** *NTRCA প্রশ্নব্যাংক*

```
for (i = 0; i < n-1; i++) {
  m = i;                    /* largest so far */
  for (j = i+1; j < n; j++)
    if (a[j] > a[m]) m = j;
  swap(a[i], a[m]);
}
```

পাস	পাসের পর অ্যারে
১	**84** 72 80 65 45 52 37
২	**84 80** 72 65 45 52 37
৩, ৪	পরিবর্তন নেই (72, 65 যথাস্থানে)
৫	**84 80 72 65 52** 45 37
৬	**84 80 72 65 52 45 37**

$n = 7$-এ সবসময় $n(n-1)/2$ = **২১টি তুলনা**, সাজানো ইনপুটেও, তাই *best = worst =* $O(n^2)$; সর্বোচ্চ $n - 1$টি *swap*, তাই সরাতে খরচ বেশি এমন ডেটায় উপযোগী।

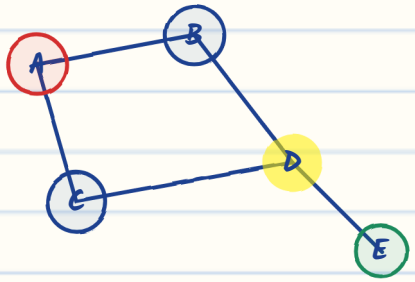
৮ BFS বনাম DFS

গ্রাফ এসেছে ১৪টি পরীক্ষায়

প্র২১ BFS ও DFS-এর পার্থক্য কী? কোন ধরনের সমস্যায় DFS-এর চেয়ে BFS ব্যবহার করা ভালো?

✎ **যে পরীক্ষায় এসেছে:** 45th BCS CSE 2024 · BPSC AP 2021 · 17th NTRCA CSE 2023 · BPSC AP 2022

বিষয়	BFS	DFS
খোঁজে	লেভেল ধরে	এক শাখার শেষ পর্যন্ত, তারপর পিছিয়ে আসে
ব্যবহার করে	**কিউ**	**স্ট্যাক** / রিকার্শন
সময়	$O(V + E)$	$O(V + E)$
মেমোরি	পুরো একটি লেভেল (চওড়া)	একটি পথ (গভীর)
যা দেয়	সংক্ষিপ্ততম পথ *(unweighted)*	সাইকেল, টপোলজিক্যাল সর্ট, SCC



A থেকে BFS: A B C D E · DFS: A B D C E

BFS ব্যবহার করুন যখন উত্তর **শুরুর কাছাকাছি** বা সবচেয়ে কম এজ দরকার: *unweighted* গ্রাফে সংক্ষিপ্ততম পথ, ২ ধাপের মধ্যে বন্ধু, গভীরতা ধরে ওয়েব ক্রলিং, *peer-to-peer* সার্চ। গভীর অনুসন্ধান, সাইকেল খোঁজা, পাজল, টপোলজিক্যাল ক্রমে DFS।

পরীক্ষার ফাঁদ একটি প্রচলিত ছাপা উত্তরে লেখা DFS স্ট্যাক ব্যবহার করে "BFS ট্রাভার্সালের জন্য" এবং "BFS ধীর": দুটোই ভুল। দুটোই $O(V + E)$; **BFS → কিউ** (MCQ ✎ **যে পরীক্ষায় এসেছে:** 49th BCS ICT 2025), **DFS → স্ট্যাক**।

প্র২২ উদাহরণসহ গ্রাফের BFS ট্রাভার্সাল ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA ICT 2023 · DBBL PO 2017

কিউ [A] → A নিন, B, C যোগ → B নিন, D যোগ → C নিন (D দেখা হয়ে গেছে) → D নিন, E যোগ → E নিন। ক্রম A B C D E; লেভেল A | B C | D | E।

প্র২৩ Depth-first search কেন optimal নয়?

✎ **যে পরীক্ষায় এসেছে:** 45th BCS ICT 2024

DFS যে পথ **প্রথমে** পায় সেটাই দেয়, সংক্ষিপ্ততমটি নয়: সস্তা A → C → D থাকলেও সে গভীরে A → B → D → E চলে যেতে পারে, আর অসীম / খুব গভীর শাখায় হয়তো ফেরেই না। BFS (সমান খরচে) এবং *uniform-cost* / A^* optimal।

প্র২৪ কোন সমস্যা *adjacency matrix*-এর চেয়ে *adjacency list*-এ বেশি দক্ষতায় সমাধান হয়, আর কোনটি উল্টো?

✎ **যে পরীক্ষায় এসেছে:** *Combined Bank AP 2023 (ছাপা ভাষা অস্পষ্ট)*

প্রয়োজন	Adjacency list	Adjacency matrix
মেমোরি	$O(V + E)$: sparse গ্রাফ (রাস্তা, সোশ্যাল)	$O(V^2)$: dense গ্রাফ
u–v এজ আছে কি?	$O(\text{degree})$	$O(1)$
সব প্রতিবেশী দেখা (BFS, DFS, Dijkstra)	$O(\text{degree})$	প্রতি ভার্টেক্সে $O(V)$
Floyd–Warshall, dense গ্রাফে Prim	ধীর	**স্বাভাবিক পছন্দ**

প্র২৫ ৭ ভার্টেক্সের একটি *complete* বাইনারি ট্রির *adjacency list* এবং সমতুল্য *adjacency matrix* দিন; ভার্টেক্সগুলো বাইনারি হিপের মতো 1 … 7 নম্বরযুক্ত।

✎ **যে পরীক্ষায় এসেছে:** *BB AP 2023*

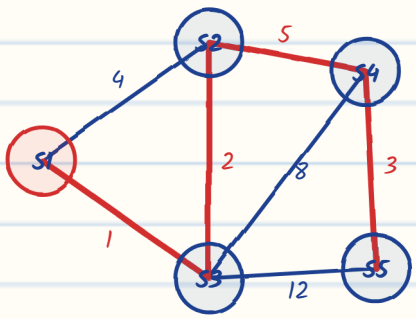
হিপ নম্বরে: i যুক্ত $2i$ ও $2i + 1$-এর সাথে। লিস্ট: $1 \to 2, 3$ · $2 \to 1, 4, 5$ · $3 \to 1, 6, 7$ · $4 \to 2$ · $5 \to 2$ · $6 \to 3$ · $7 \to 3$। ম্যাট্রিক্স প্রতিসম (undirected), ১২টি ১ = ২ × ৬টি এজ।

	1	2	3	4	5	6	7
1	0	1	1	0	0	0	0
2	1	0	0	1	1	0	0
3	1	0	0	0	0	1	1
4	0	1	0	0	0	0	0
5	0	1	0	0	0	0	0
6	0	0	1	0	0	0	0
7	0	0	1	0	0	0	0

৯ সংক্ষিপ্ততম পথ: ডাইক্স্ট্রা (গ্রিডি) *45th BCS · NESCO*

প্র২৬ *Feasible* ও *optimal solution* সংজ্ঞায়িত করুন। *Single-source shortest path*-এর গ্রিডি অ্যালগরিদম লিখুন। / *S1 … S5* পাঁচটি নোড, লাইনের মান বিদ্যুৎ পাঠানোর খরচ: সংক্ষিপ্ততম পথ ও গড় খরচ বের করুন।

✎ **যে পরীক্ষায় এসেছে:** *45th BCS CSE 2024 · NESCO AM(ICT) 2018 (গ্রাফ ছাপা নেই: নিচে নমুনা)*



Feasible = সব শর্ত মানে (যেকোনো $S1 \to S5$ পথ); ***optimal*** = *feasible*-গুলোর মধ্যে সেরা মানেরটি (সবচেয়ে সস্তা)।

1. $d[S1] = 0$, বাকিগুলো ∞; না-দেখা ভার্টেক্সের মধ্যে **সবচেয়ে ছোট d** বাছুন
2. তার এজ *relax* করুন: $d[v] = \min(d[v], d[u] + w)$
3. $S1$ 0 → $S3$ 1 → $S2$ 3 ($S3$ হয়ে) → $S4$ 8 → $S5$ 11 ($S4$ হয়ে; $S3$ হয়ে হতো 1 + 12 = 13)

→ $S2 = 3$ ($S1 \to S3 \to S2$), $S3 = 1$, $S4 = 8$ ($S1 \to S3 \to S2 \to S4$), $S5 = 11$ ($S1 \to S3 \to S2 \to S4 \to S5$) · গড় খরচ $(3 + 1 + 8 + 11) / 4 = \mathbf{5.75}$। হিপসহ সময় $O((V + E) \log V)$; ঋণাত্মক এজ চলবে না (তখন Bellman–Ford)।

Adjacency উপস্থাপন থেকে weighted গ্রাফ আঁকা ✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. April 2017 · BUET M.Sc. May 2016 (উপস্থাপনটি ছাপা নেই): প্রতিটি ভার্টেক্সে একটি বৃত্ত, প্রতিটি অশূন্য ঘরে একটি লাইন (প্রতিসম হলে একবার, না হলে তীর), ঘরের মান লাইনের ওপর লেখা; যেমন পৃ. ১০-এর S1 … S5 গ্রাফের লিস্ট S1: S2/4, S3/1 · S2: S3/2, S4/5 · …

১০ Minimum spanning tree: ক্রুসকাল

RAKUB Programmer 2021

প্র২৭ Kruskal's অ্যালগরিদম দিয়ে গ্রাফটির Minimum Spanning Tree বের করুন।
✎ **যে পরীক্ষায় এসেছে:** RAKUB Programmer 2021 (গ্রাফ ছাপা নেই; টেবিলে নমুনা)

ওজন অনুযায়ী সাজানো এজ	নেবেন?
BC 1 · BD 2 · DE 2 · AC 3	**সবগুলো নিন** (সাইকেল নেই)
AB 4 · CD 4 · CE 5	বাদ: সাইকেল তৈরি করবে
EF 6	**নিন** → ৬ ভার্টেক্সে ৫টি এজ, থামুন

MST = BC, BD, DE, AC, EF, মোট **14**।
Kruskal: এজ সাজান, যে সবচেয়ে সস্তা এজ দুটি আলাদা অংশ জোড়ে সেটি নিন (union-find); O(E log E) ✎ **যে পরীক্ষায় এসেছে:** MCQ: কোচিং MCQ সেট, পরীক্ষার ট্যাগ নেই। Prim একটি ভার্টেক্স থেকে একটিই ট্রি বাড়ায়।

Suffix tree MCQ ✎ **যে পরীক্ষায় এসেছে:** 49th BCS ICT 2025: একটি স্ট্রিংয়ের সব suffix-এর সংকুচিত trie → **দ্রুত স্ট্রিং (substring) সার্চ**, O(m)-এ।

১১ অ্যারে বনাম লিংকড লিস্ট

৫টি পরীক্ষা ✦ সবচেয়ে বেশি আসা প্রশ্ন

প্র২৮ Linked list ও Array-এর মধ্যে পার্থক্য লিখুন। অ্যারের বদলে লিংকড লিস্ট ব্যবহারের সুবিধা ও অসুবিধা লিখুন।
✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA ICT 2023 · BNPS AP 2017 · BPSC AP 2021 · BPSC AP 2023 · BPSC ME 2021 · সুবিধা / অসুবিধা: 45th BCS CSE 2024 · ব্যাংক আইটি ভাইভা

বিষয়	অ্যারে	লিংকড লিস্ট
মেমোরি	**পাশাপাশি**, স্থির আকার	ছড়ানো নোড, বাড়ে / কমে
k-তম উপাদান	**O(1)** ইনডেক্সে	O(n), head থেকে হাঁটতে হয়
যোগ / মোছা (অবস্থান জানা)	O(n), উপাদান সরাতে হয়	**O(1)**, পয়েন্টার বদল
বাড়তি জায়গা	নেই	প্রতি নোডে একটি পয়েন্টার
ক্যাশ, binary search	ক্যাশ-বান্ধব; binary search চলে	লোকালিটি কম; দ্রুত binary search নেই

মনে রাখার কৌশল **"Array = Address, List = Links"**: অ্যারে জেতে **ইনডেক্সে পড়ায়**, লিস্ট জেতে **যোগ ও মোছায়**। NESCO-র প্রশ্ন "Insert, Delete না Search?" ✎ **যে পরীক্ষায় এসেছে:** NESCO AM(ICT) 2021 → লিংকড লিস্ট ভালো **যোগ ও মোছায়**, খারাপ সার্চে।

প্র২৯ *Linked list* কী? একটি লিংকড লিস্ট তৈরির অ্যালগরিদম দিন, উদাহরণ চিত্রে দেখান, এবং এর প্রকারভেদ চিত্রসহ বর্ণনা করুন।

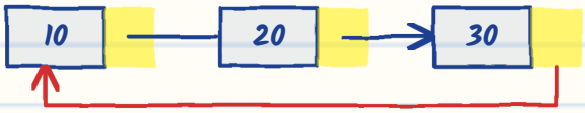
✎ **যে পরীক্ষায় এসেছে:** *17th NTRCA CSE 2023 · BPSC AE 2022 · 18th NTRCA College ICT 2024 · singly, doubly, circular: BNPS AP 2017*



singly: data | next



doubly: prev | data | next



circular: শেষ → প্রথম

```
struct Node { int data; struct Node *next; };
head = NULL;                          /* create 10 → 20 → 30 */
for each value x (taken in reverse: 30, 20, 10):
    n = malloc(sizeof *n);
    n->data = x; n->next = head;  /* insert at front */
    head = n;
```

লিংকড লিস্ট হলো **নোডের** রৈখিক সংগ্রহ, প্রতিটি নোডে ডেটা ও পরের নোডের পয়েন্টার; ***head*** দিয়ে শুরু, শেষে *NULL*। *Doubly* লিস্ট পেছনের দিকেও পয়েন্টার রাখে (উল্টো হাঁটা, নোড দিলে মোছা সহজ); *circular* লিস্টে শেষ নোড প্রথমটির সাথে যুক্ত (*round-robin* কিউ)।

প্র৩০ সাজানো লিংকড লিস্টে কি *binary search* প্রয়োগ করা যায়? উদাহরণসহ যুক্তি দিন।

✎ **যে পরীক্ষায় এসেছে:** *45th BCS CSE 2024*

সম্ভব, কিন্তু **দক্ষ নয়**: মাঝখানে পৌঁছাতে *n/2*টি নোড হাঁটতে হয়, তাই মোট *O(n)*, *linear search*-এর চেয়ে ভালো নয়। *3 → 8 → 15 → 21 → 34*-এ মাঝখান (*15*) পেতে ২ ধাপ, পরের মাঝখানে আরও ১... ***Skip list*** বা *BST* দেয় *O(log n)*।

প্র৩১ *3, 5, 7, 11, 13, 17* তালিকাটি ক্রমানুসারে রাখা। *11* সরাতে *queue, heap, linked list, stack*-এর কোনটিতে সবচেয়ে কম ডেটা সরাতে হয়? যুক্তি দিন।

✎ **যে পরীক্ষায় এসেছে:** *38th BCS 2018*

লিংকড লিস্ট: *7*-এর *next*-কে *13*-তে দিন, কিছুই সরাতে হয় না। স্ট্যাক / কিউকে বাকিগুলো *pop* ও *push* করতে হয়; অ্যারে *13, 17* সরায়; হিপ আবার *heapify* করে।

প্র৩২ লিংকড লিস্টে কি *divide and conquer* প্রয়োগ করা যায়?

✎ **যে পরীক্ষায় এসেছে:** ব্যাংক আইটি ভাইভা

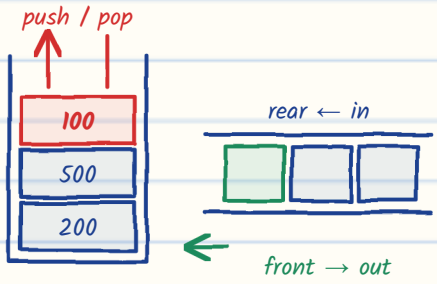
হ্যাঁ: লিংকড লিস্টের স্বাভাবিক সর্ট *merge sort*। *slow* ও *fast* পয়েন্টার দিয়ে মাঝখানে ভাগ (*O(n)*), দুই অর্ধেক সর্ট, নোড জুড়ে মার্জ: *O(n log n)*, বাড়তি অ্যারে লাগে না। *Binary search*-এ লাভ নেই, কারণ মাঝখানে পৌঁছাতেই *O(n)* লাগে।

```
slow = head; fast = head->next;
while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}  /* slow = end of first half */
```

১২ স্ট্যাক বনাম কিউ ১০টি পরীক্ষা + ভাইভা

প্র৩৩ Stack ও Queue-এর মধ্যে পার্থক্য লিখুন, এবং প্রতিটি দিয়ে সমাধান হয় এমন দুটি সমস্যা লিখুন।
✎ **যে পরীক্ষায় এসেছে:** BPSC Lecturer 2017 · Combined Bank AP 2024 · DESCO SAE 2023 · 18th NTRCA College ICT 2024 · ব্যাংক আইটি ভাইভা

বিষয়	Stack	Queue
ক্রম	LIFO	FIFO
ব্যবহৃত প্রান্ত	একটি (top)	দুটি (rear-এ ঢোকে, front থেকে বের হয়)
অপারেশন	push, pop, peek	enqueue, dequeue, front
সমস্যা	ফাংশন কল / রিকার্শন, undo, বন্ধনী মেলানো, postfix মূল্যায়ন	CPU / প্রিন্টার শিডিউলিং, BFS, বাফার, কল সেন্টারের লাইন



প্র৩৪ Push(200), Push(500), Push(100), S = Pop() । S কত?
✎ **যে পরীক্ষায় এসেছে:** BAPEX AGM(ICT) 2023
→ S = 100 (শেষে ঢোকে, আগে বের হয়); এখন top 500

প্র৩৫ Stack-এর operation গুলো সংক্ষেপে বর্ণনা করুন। আমরা কেন stack ব্যবহার করি?
✎ **যে পরীক্ষায় এসেছে:** BPSC SAE 2021 · BNPS AP 2017 · push ও pop (MCQ): BREB 2019 · PKB Programmer 2019
push: top++ তারপর a[top] = x (ভরা হলে overflow) · **pop**: x = a[top--] (খালি হলে underflow) · **peek**, **isEmpty**: সব O(1)। যখনই **সর্বশেষ** উপাদান আগে সামলাতে হয় তখন ব্যবহৃত।

প্র৩৬ কিউয়ের Enqueue অপারেশনের ধাপগুলো লিখুন।
✎ **যে পরীক্ষায় এসেছে:** Combined Bank Officer(IT) 2020

১. কিউ ভরা হলে (rear = MAX − 1; circular: (rear + 1) % MAX = front) → **overflow**, থামুন
২. খালি হলে (front = −1) front = 0 করুন
৩. rear = rear + 1 (circular: (rear + 1) % MAX)
৪. queue[rear] = item; O(1)-এ শেষ

প্র৩৭ একটি circular queue-তে ৬টি ঘর (0 – 5), শুরুতে খালি। A, C, D যোগ; F চোকান; দুটি মুছুন; K, L, M যোগ; দুটি মুছুন; R যোগ। প্রতি ধাপে FRONT ও REAR দিন।

✎ **যে পরীক্ষায় এসেছে:** 38th BCS 2018

ধাপ	শুরু	+A C D	+F	−2	+K L M	−2	+R
FRONT	−1	0	0	2	2	4	4
REAR	−1	2	3	3	0 (M ঘুরে আসে)	0	1

→ শেষে FRONT = 4, REAR = 1; কিউতে আছে L (4), M (0), R (1)। ঘুরে আসাটাই circular queue-এর মূল বিষয়।

প্র৩৮ শুধু দুটি স্ট্যাক দিয়ে একটি FIFO কিউ বানানোর যুক্তি ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** ব্যাংক AP লিখিত (শিটে ব্যাংকের নাম কাটা)

enqueue(x): x-কে **in**-এ push। **dequeue():** **out** খালি হলে in থেকে সব pop করে out-এ push (এতে ক্রম উল্টে যায়), তারপর out থেকে pop। প্রতিটি উপাদান সর্বোচ্চ দুবার সরে → **amortised O(1)**। enq 1, 2, 3; deq → 1; enq 4; deq → 2, 3, 4। উল্টো প্রশ্ন (MCQ): স্ট্যাক বানাতে লাগে **দুটি কিউ**।

১৩ হিপ ৮টি পরীক্ষা

প্র৩৯ Max heap-এর বৈশিষ্ট্যগুলো এবং heapsort অ্যালগরিদম লিখুন।

✎ **যে পরীক্ষায় এসেছে:** BPSC SCO 2022

✓ **আকার:** একটি complete বাইনারি ট্রি (লেভেলগুলো পূর্ণ, শেষটি বাম থেকে) → অ্যারেতে রাখা যায়

✓ **ক্রম:** প্যারেন্ট ≥ প্রতিটি সন্তান, তাই **সর্বোচ্চটি রুটে**

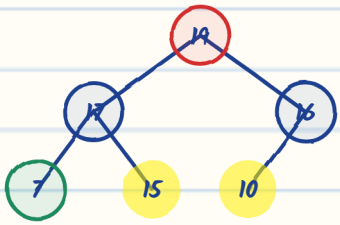
✓ উচ্চতা $\lfloor \log_2 n \rfloor$: insert ও delete-max O(log n), find-max O(1)

✓ ভাইবোনের মধ্যে ক্রম নেই (এটি BST নয়)

```
heapSort(A, n):
  for i = n/2 - 1 down to 0:      // build: O(n)
      siftDown(A, n, i)
  for end = n-1 down to 1:
      swap(A[0], A[end])           // max to the back
      siftDown(A, end, 0)          // O(log n)
// total O(n log n), in place, not stable
```

প্র৪০ {15, 19, 10, 7, 17, 16} থেকে একটি হিপ তৈরি করুন এবং 7 মোছার পর আবার আঁকুন।

✎ **যে পরীক্ষায় এসেছে:** PGCB AE(CSE) 2021



১ একে একে যোগ, **sift up:** 15 · 19 ওপরে ওঠে · 10 · 7 · 17 পার হয় 15 · 16 পার হয় 10

২ 7 মোছা: শেষ উপাদান (10) তার জায়গায়, sift: 10 < প্যারেন্ট 17, সন্তান নেই → যেখানে আছে

৩ ফল 19 / 17, 16 / 10, 15

একে একে যোগ করে max-heap: 19 17 16 7 15 10

ছাপা সমাধানে হিপ নয়, BST আঁকা। Min-heap বোঝালে: 7 / 10, 15 / 19, 17, 16; রুট 7 মুছলে 10 / 16, 15 / 19, 17।

প্রশ্ন ১ 40, 45, 80, 90, 50, 70 আইটেমগুলো দিয়ে হিপ ও বাইনারি সার্চ ট্রি আঁকুন।

✎ **যে পরীক্ষায় এসেছে:** SGFL AE(IT) 2023

একে একে যোগ করে max-heap: **90 / 80, 70 / 40, 50, 45** (bottom-up heapify দেয় 90 / 50, 80 / 45, 40, 70: এটিও বৈধ)

Heap sort আউটপুট: 40 45 50 70 80 90

BST: 40 → ডানে 45 → ডানে 80 → (50 → ডানে 70, 90)

inorder 40 45 50 70 80 90 = সাজানো ✓

উচ্চতা ৪: অসমতল, প্রায় একপাশে হেলানো ট্রি

হিপ সর্ট বনাম মার্জ সর্ট ✎ যে পরীক্ষায় এসেছে: BPSC AP 2021 · BBL AM(ICT) 2019	Heap sort	Merge sort	Insertion sort
সময় (worst)	n log n	n log n	n^2 (প্রায় সাজানো হলে n)
বাড়তি জায়গা	**O(1)**	O(n)	O(1)
Stable	না	**হ্যাঁ**	হ্যাঁ
সুবিধা	in place + নিশ্চিত	stable, লিংকড লিস্ট, বাহ্যিক ডেটা	সহজ, ছোট / সাজানো ইনপুটে দ্রুত

মনে রাখার কৌশল **"Heap = Hierarchy, BST = Bisection"**: হিপ শুধু প্যারেন্ট ≥ সন্তান নিশ্চিত করে (সর্বোচ্চটির জন্য ভালো); BST রাখে বাম < রুট < ডান (যেকোনো কী খোঁজার জন্য ভালো)।

১৪ লিনিয়ার ও বাইনারি সার্চ ৮টি পরীক্ষা

প্রশ্ন ২ Linear Search ও Binary Search-এর মধ্যে পার্থক্য লিখুন। Binary search অ্যালগরিদম ধাপে ধাপে ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE 2023 · BEP2A Programmer 2023 · ধাপে ধাপে: ব্যাংক AP লিখিত (নাম কাটা)

বিষয়	Linear	Binary
ডেটা	যেকোনো ক্রম	**সাজানো**, random access
ধারণা	একে একে দেখা	পরিসর অর্ধেক করা
Best / worst	O(1) / O(n)	O(1) / **O(log n)**
10^6টি আইটেম	10^6 পর্যন্ত যাচাই	≤ ২০টি যাচাই

```
lo = 0; hi = n - 1;
while (lo <= hi) {
  mid = lo + (hi - lo) / 2;
  if (a[mid] == key) return mid;
  if (a[mid] < key) lo = mid + 1;
  else              hi = mid - 1;
}
return -1;
```

→ 3 8 15 21 34 42 57-এ 34 খোঁজা: mid 3 (21) → ডানে · mid 5 (42) → বামে · mid 4 (34) পাওয়া গেল, **৩টি যাচাই**

প্রশ্ন ৩৩ Binary search-এর recurrence সমীকরণ লিখুন এবং সমাধান করুন।

✎ **যে পরীক্ষায় এসেছে:** Petrobangla AM(IT) 2022 · জটিলতা (MCQ): Combined Bank Officer(IT) 2018

একটি তুলনা, তারপর একটি অর্ধেক: $T(n) = T(n/2) + c$, $T(1) = c$। খুলে লিখলে: $T(n) = T(n/2^k) + kc$; $n/2^k = 1$ হয় $k = \log_2 n$-এ → $T(n) = c \log_2 n + c =$ **$O(\log n)$**। (ছাপা সমাধানে merge sort-এর $2T(n/2) + n$ সমাধান করা।)

প্রশ্ন ৩৪ Linear Search অ্যালগরিদম কী? এই অ্যালগরিদমের best case ও worst case complexity বর্ণনা করুন।

✎ **যে পরীক্ষায় এসেছে:** BPSC SAE 2021

a[0], a[1], ... দেখতে থাকুন যতক্ষণ না key মেলে বা অ্যারে শেষ হয়। Best: key a[0]-এ, **১টি তুলনা, O(1)**। Worst: শেষে বা নেই, **nটি তুলনা, O(n)**। গড় $(n + 1)/2$। অসাজানো ডেটা ও লিংকড লিস্টে চলে।

১৫ হ্যাশিং

BCC · 49th BCS

প্রশ্ন ৩৫ কোন ডেটা স্ট্রাকচার একগুচ্ছ অবজেক্টে $O(1)$ প্রত্যাশিত সার্চ টাইম দেয়?

✎ **যে পরীক্ষায় এসেছে:** BCC AP 2021 (ছাপা "try yourself") · chaining MCQ: 49th BCS ICT 2025

হ্যাশ টেবিল: h(key) সরাসরি ঘর দেয়। Collision সামলানো হয় **chaining** (প্রতি ঘরে একটি লিস্ট) বা open addressing (linear / quadratic probing, double hashing) দিয়ে। Load factor $\alpha = n/m$ হলে chaining-এ গড় সার্চ **$O(1 + \alpha)$**। টেবিল বড় করে $\alpha \le 1$ রাখলে $O(1)$। সব একঘরে পড়লে worst case $O(n)$।

$h(k) = k \bmod 7$, chaining
0: 14 → 21
1:
3: 10 → 24
5: 5

প্রশ্ন ৩৬ "Blind search" ও "Intelligent search"-এর তুলনা করুন। Heuristic function কী?

✎ **যে পরীক্ষায় এসেছে:** 45th BCS ICT 2024

Blind (uninformed)	Intelligent (informed, heuristic)
লক্ষ্য কত কাছে জানে না	লক্ষ্য পর্যন্ত দূরত্বের একটি অনুমান ব্যবহার করে
BFS, DFS, uniform-cost, iterative deepening	greedy best-first, **A***, hill climbing
অনেক নোড ঘাঁটে; সহজ	heuristic ভালো হলে অনেক কম নোড

Heuristic function $h(n)$ নোড n থেকে লক্ষ্যের খরচ অনুমান করে, যেমন মানচিত্রে সরলরেখার দূরত্ব বা 8-puzzle-এ ভুল জায়গার টাইল। A^* ব্যবহার করে $f(n) = g(n) + h(n)$, এবং h কখনো বাড়িয়ে না বললে (**admissible**) এটি optimal।

১৬ Divide and conquer বনাম dynamic programming

১২টি পরীক্ষা

প্রশ্ন ৭ Divide and Conquer ও Dynamic Programming দুটোই সমস্যাকে ছোট অংশে ভেঙে সমাধান করে। পার্থক্য কী? DP কীভাবে D&C-এর সাথে সম্পর্কিত?

✎ **যে পরীক্ষায় এসেছে:** BCC AP 2021 · W2PGCL AE(CSE) 2023 · সম্পর্ক: BPSC AP 2023

বিষয়	Divide and conquer	Dynamic programming
উপ-সমস্যা	**স্বাধীন** (আলাদা অর্ধেক)	**ওভারল্যাপিং** (একই বারবার আসে)
পুনরাবৃত্ত কাজ	বারবার এলে আবার সমাধান	প্রতিটি একবার সমাধান, সংরক্ষণ করে (টেবিল / মেমো)
দিক	top-down রিকার্শন	bottom-up টেবিল বা মেমোসহ top-down
উদাহরণ	merge sort, quick sort, binary search	Fibonacci, LCS, 0/1 knapsack, Floyd-Warshall

সম্পর্ক: DP হলো divide and conquer **সাথে মেমোরি**: D&C-এর রিকার্শন ট্রিতে একই উপ-সমস্যা বহুবার এলে তার উত্তর রেখে দিলে exponential সময় polynomial হয়। MCQ "কোনগুলো divide and conquer ব্যবহার করে?" ✎ **যে পরীক্ষায় এসেছে:** BPDB AE(CSE) 2021 (অপশন ছাপা নেই) → merge sort, quick sort, binary search, Strassen।

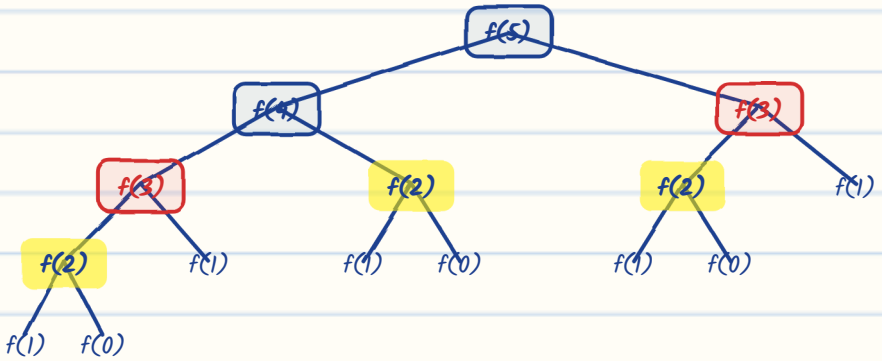
প্রশ্ন ৮ Dynamic programming দিয়ে Fibonacci সংখ্যা নির্ণয়ের অ্যালগরিদম লিখুন।

✎ **যে পরীক্ষায় এসেছে:** BPSC ADA 2022 · PSB Programmer 2018

```
long fib(int n) {
    long f[n + 2];
    f[0] = 0; f[1] = 1;
    for (int i = 2; i <= n; i++)
        f[i] = f[i-1] + f[i-2];
    return f[n];
}
```

f: 0 1 1 2 3 5 8 13 21 34 **55** → fib(10) = 55

সাধারণ রিকার্শন fib(n − 1) + fib(n − 2) একই মান বারবার হিসাব করে: **$O(2^n)$**। টেবিল প্রতিটি একবার হিসাব করে: **O(n) সময়**, O(n) মেমোরি (শেষ দুটি রাখলে O(1))।



সাধারণ রিকার্শন: fib(5)-এ ১৫টি কল; f(3) দুবার (লাল), f(2) তিনবার (হলুদ) সমাধান হয়। DP টেবিল f(0) … f(5) প্রতিটি একবার সমাধান করে।

মনে রাখার কৌশল DP-র দুটি শর্ত: **ওভারল্যাপিং উপ-সমস্যা** (ওপরের পুনরাবৃত্ত বাক্স) ও **optimal substructure**। মেমোসহ top-down = **memoization**; টেবিলসহ bottom-up = **tabulation**।

১৭ গ্রিডি পদ্ধতি NTRCA · BPSC

প্রশ্ন Greedy Method ও Dynamic Algorithm-এর মধ্যে পার্থক্য লিখুন। গ্রিডি অ্যালগরিদম কি সবসময় optimal সমাধান দেয়? না দিলে কখন দেয়?
✎ **যে পরীক্ষায় এসেছে:** 18th NTRCA College ICT 2024 · optimality: BPSC AP 2023

Greedy	Dynamic programming
এখন যেটা সেরা মনে হয়, আর ফেরা নেই	সব উপ-পছন্দ চেষ্টা করে সেরাটি রাখে
দ্রুত, কম মেমোরি	ধীর, একটি টেবিল
কিছু সমস্যাতেই optimal	সবসময় optimal (সমীকরণ ঠিক হলে)
Dijkstra, Kruskal, Huffman, fractional knapsack	0/1 knapsack, LCS, matrix chain

সবসময় optimal নয় কয়েন {1, 3, 4}, পরিমাণ 6: গ্রিডি নেয় 4 + 1 + 1 (৩টি কয়েন), optimal 3 + 3 (২টি)। গ্রিডি optimal হয় যখন সমস্যায় **greedy-choice property** ও **optimal substructure** থাকে (যেমন fractional knapsack, MST, activity selection)।

প্রশ্ন Huffman coding-এর time complexity কত?
✎ **যে পরীক্ষায় এসেছে:** Combined Bank Officer(IT) 2019

গ্রিডি: সব প্রতীক ফ্রিকোয়েন্সি অনুযায়ী একটি **min-heap**-এ রাখুন; বারবার সবচেয়ে ছোট দুটিকে একটি নতুন নোডের (যোগফল) নিচে জুড়ুন। $n - 1$টি মার্জ × $O(\log n)$ হিপ কাজ → $O(n \log n)$। ফ্রিকোয়েন্সি a 5, b 9, c 12, d 13, e 16, f 45: মার্জ a+b = 14, c+d = 25, 14+e = 30, 25+30 = 55, f+55 = 100।

প্রতীক	f	c	d	a	b	e
কোড	0	100	101	1100	1101	111

→ ২২৪ বিট, স্থির ৩-বিট কোডে ৩০০; কোনো কোড অন্যটির prefix নয়।

প্রশ্ন nজন গ্রাহক সময়-স্লট [start, end] চায়; ওভারল্যাপ করা দুটি অনুরোধ একসাথে মেটানো যায় না। সর্বোচ্চ সংখ্যক গ্রাহককে সেবা দেওয়ার অ্যালগরিদম লিখুন।
✎ **যে পরীক্ষায় এসেছে:** BPDB AE(CSE) 2023

```
sort requests by END time
last = -infinity; count = 0
for each [s, e] in that order:
    if s >= last:            // no overlap
        serve it; count++; last = e
return count                 // O(n log n)
```

[1,4] [3,5] [0,6] [5,7] [3,9] [5,9] [6,10] [8,11] → সেবা **[1,4] [5,7] [8,11]** = ৩ জন গ্রাহক

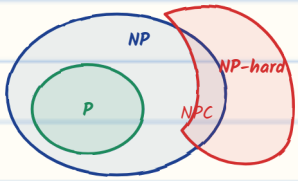
পরীক্ষার ফাঁদ **start** সময় অনুযায়ী সাজানো (যেমন একটি ছাপা সমাধানে) ভুল: [1,10], [2,3], [4,5] → শুধু [1,10] সেবা পায়, অথচ [2,3] ও [4,5] দিলে ২ জন। সবচেয়ে আগে **শেষ হওয়া** স্লট বেশি জায়গা রাখে: ক্লাসিক activity-selection গ্রিডি।

১৮ ব্যাকট্র্যাকিং ও NP

45th BCS CSE

প্র৫২ উদাহরণসহ সংজ্ঞা দিন: *Backtracking*, *NP*, *NP-Hard* ও *NP-complete problem*।
✎ **যে পরীক্ষায় এসেছে:** *45th BCS CSE 2024*

শব্দ	অর্থ	উদাহরণ
Backtracking	ধাপে ধাপে সমাধান গড়ে, নিয়ম ভাঙা ধাপ বাতিল করে পিছিয়ে যায়	*N-queens, sudoku*
P	*polynomial* সময়ে সমাধানযোগ্য	সর্টিং, সংক্ষিপ্ততম পথ
NP	একটি সমাধান *polynomial* সময়ে **যাচাই** করা যায়	*SAT, subset sum*
NP-hard	প্রতিটি *NP* সমস্যা এতে রিডিউস হয়; *NP*-তে থাকা জরুরি নয়	*TSP optimisation, halting*
NP-complete	*NP*-তে **এবং** *NP-hard*	*SAT, 3-colouring, TSP decision*



যদি $P \neq NP$

১৯ গেম সার্চ ও অন্যান্য শব্দ

BCS · BUET M.Sc.

প্র৫৩ *Minimax ("maximum")* অ্যালগরিদম কী? গেম খেলায় *'Terminal'* ও *'Utility'* সংজ্ঞায়িত করুন। অ্যালগরিদমটির সীমাবদ্ধতা কী?
✎ **যে পরীক্ষায় এসেছে:** *45th BCS ICT 2024 (ছাপা "maximum algorithm")*

Minimax গেম ট্রি খোঁজে: *MAX* সবচেয়ে বড় মানের সন্তান বাছে, *MIN* সবচেয়ে ছোট, মানগুলো লিফ থেকে ওপরে তোলা হয়। ***Terminal*** অবস্থা = খেলা শেষ (জয়, পরাজয়, ড্র); ***utility*** = তার সংখ্যাগত ফল (*+1, 0, −1*)। **সীমাবদ্ধতা:** পুরো ট্রি ঘাঁটে, $O(b^m)$ (দাবায় $b \approx 35$, $m \approx 100$): বাস্তবে অসম্ভব → ***alpha-beta pruning*** ও মূল্যায়ন ফাংশনসহ গভীরতার সীমা দিয়ে কাটা হয়।

IDA* ✎ **যে পরীক্ষায় এসেছে:** *BUET M.Sc. April 2017*: *iterative-deepening A**: $f = g + h$ খরচের একটি সীমা নিয়ে *depth-first*, প্রতি রাউন্ডে সীমা বাড়ে। সুবিধা: *admissible h*-এ *A**-এর মতো *optimal*, কিন্তু মেমোরি *A**-এর পুরো *frontier*-এর বদলে শুধু ***O(depth)***।

Parameterized algorithm ✎ **যে পরীক্ষায় এসেছে:** *BUET M.Sc.*: *(complexity theory)* চলে $f(k) \cdot n^{O(1)}$ সময়ে, *exponential* শুধু একটি ছোট প্যারামিটার *k*-তে, যেমন *vertex cover* $O(2^k n)$-এ। কোডে প্যারামিটারের (টাইপ, *comparator*) ওপর অ্যালগরিদম লিখলে একটি রুটিন অনেক ইনপুটে কাজ করে: কপি ছাড়াই পুনর্ব্যবহার।

২০ Infix, prefix, postfix BTCL · BBL

প্রশ্ন ৪ Infix, Prefix, Postfix notation ব্যাখ্যা করুন, এবং infix রূপান্তরের পদ্ধতি বর্ণনা করুন।
✎ **যে পরীক্ষায় এসেছে:** BTCL JAM 2022 · BBL AM(ICT) 2019 · এক্সপ্রেশনগুলো NTRCA প্রশ্নব্যাংক থেকে

Infix A + B (অপারেটর মাঝে, বন্ধনী লাগে) · **prefix** (Polish) + A B · **postfix** (reverse Polish) A B +। Pre/postfix-এ **বন্ধনী লাগে না**, একটি স্ট্যাকেই মূল্যায়ন হয়।

রূপান্তর: বাম থেকে ডানে পড়ুন; অপারেন্ড → আউটপুট; (→ push;) → (পর্যন্ত pop; অপারেটর → top-এর precedence বেশি (বা সমান, বাম-associative) হলে pop, তারপর push; শেষে সব pop। ↑ ডান-associative।

Infix	Postfix	Prefix
A + B * C	A B C * +	+ A * B C
(A + B) * C	A B + C *	* + A B C
A ↑ B * C − D + E / F / (G + H)	AB↑C*D−EF/GH+/+	+ −*↑ABCD//EF+GH
(A + B) * C − (D − E) ↑ (F + G)	AB+C*DE−FG+↑−	−*+ABC↑−DE+FG
A − B / (C * D ↑ E)	ABCDE↑*/−	−A/B*C↑DE

স্ট্যাক ট্রেস: A − B / (C * D ↑ E) → postfix (NTRCA সেটের তৃতীয় এক্সপ্রেশন):

টোকেন	স্ট্যাক	আউটপুট
A		A
−	−	A
B	−	AB
/	− /	AB
(	− / (	AB
C	− / (	ABC
*	− / (*	ABC
D	− / (*	ABCD
↑	− / (* ↑	ABCD
E	− / (* ↑	ABCDE
)	− /	ABCDE↑*
শেষ		ABCDE↑*/−

২১ রিকার্শন 17th NTRCA

প্রশ্ন ৫ Tower of Hanoi সমস্যাটি সমাধানের জন্য একটি recursive অ্যালগরিদম লিখুন।
✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA ICT 2023

```
hanoi(n, from, via, to):
  if n == 0: return
  hanoi(n-1, from, to, via)
  move disk n: from → to
  hanoi(n-1, via, from, to)
```

চাল $M(n) = 2M(n-1) + 1 = 2^n - 1$। n = 3: A→C A→B C→B A→C B→A B→C A→C (৭টি চাল)।

২২ ডেটা স্ট্রাকচার ও অ্যালগরিদম NTRCA · BPSC · ব্যাংক

প্র৫৬ ডাটা স্ট্রাকচার কী? *Linear* এবং *non-linear data structures* উদাহরণসহ ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** *17th NTRCA ICT 2023 · PSB ADA 2018*

ডেটা স্ট্রাকচার হলো মেমোরিতে ডেটা সাজানোর এমন উপায় যাতে দক্ষভাবে পড়া ও বদলানো যায়। ***Linear***: উপাদানগুলো ক্রমানুসারে, পরে একটিই (অ্যারে, লিংকড লিস্ট, স্ট্যাক, কিউ)। ***Non-linear***: শ্রেণিবিন্যাস বা জাল, পরে একাধিক (ট্রি, গ্রাফ, হিপ)।

প্র৫৭ *Algorithm* কী? *Algorithm* প্রকাশের তিনটি পদ্ধতির নাম লিখুন। অ্যালগরিদমের জটিলতা কোন কোন বিষয়ের ওপর নির্ভর করে বর্ণনা করুন।

✎ **যে পরীক্ষায় এসেছে:** *BPSC AP 2021* · নির্ভরশীল বিষয়: *Combined Bank Officer(IT) 2020*

ইনপুটকে আউটপুটে রূপান্তর করে থেমে যাওয়া দ্ব্যর্থহীন ধাপের সসীম ক্রম। প্রকাশ: **সাধারণ ভাষায় ধাপ, *pseudo-code*, *flowchart*** (বা প্রোগ্রাম)। জটিলতা নির্ভর করে **ইনপুটের আকার**, ইনপুটের বিন্যাস (*best / worst / average*), গোনা অপারেশন, ডেটা স্ট্রাকচার, এবং যন্ত্র / কম্পাইলার (ধ্রুবক গুণক)-এর ওপর।

MCQ "কোনটি সবচেয়ে দ্রুত চলে?" ✎ **যে পরীক্ষায় এসেছে:** *49th BCS ICT 2025*: $O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) \rightarrow$ ***O(log n)***।

প্র৫৮ $A = [-2, 1, -3, -1, 2, 1, -5, 4]$-এর সর্বোচ্চ যোগফলের ধারাবাহিক *subarray* বের করুন।

✎ **যে পরীক্ষায় এসেছে:** *BPDB AE(CSE) 2023*

Kadane: $cur = \max(x, cur + x)$, $best = \max(best, cur)$; $O(n)$।

→ সর্বোচ্চ যোগফল **4** (*[4]*); একটি শিটের মতো $A[3] = 1$ হলে: $[1, 2, 1] = 4$

একটি ছাপা উত্তর ক্লাসিক অ্যারের $[4, -1, 2, 1] = 6$ কপি করেছে, যা এই A-তে নেই।

প্র৫৯ *int equilibrium(int arr[], int n)* লিখুন: এমন ইনডেক্স যার আগের উপাদানগুলোর যোগফল পরের উপাদানগুলোর যোগফলের সমান, না থাকলে -1। $\{-7, 1, 5, 2, -4, 3, 0\} \rightarrow 3$।

✎ **যে পরীক্ষায় এসেছে:** *BPDB AE(CSE) 2023*

```
int equilibrium(int a[], int n) {
  int total = 0, left = 0;
  for (int i = 0; i < n; i++) total += a[i];
  for (int i = 0; i < n; i++) {
    if (left == total - left - a[i]) return i;
    left += a[i];
  }
  return -1;            /* O(n), not O(n²) */
}
```

→ 3 ($-1 = -1$) · $\{1, 2, 3\} \rightarrow -1$

➜ কোন কাজে কোন স্ট্রাকচার

কাজ	স্ট্রাকচার	কাজ	স্ট্রাকচার
undo, রিকার্শন, বন্ধনী	স্ট্যাক	শিডিউলিং, *BFS*, বাফারিং	কিউ
ঘনঘন যোগ / মোছা	লিংকড লিস্ট	অবস্থান ধরে পড়া	অ্যারে
প্রায়োরিটি / দ্রুত সর্বোচ্চ	হিপ	সাজানো সার্চ, *range*	*BST / B+ tree*
নির্দিষ্ট কী-তে $O(1)$ খোঁজা	হ্যাশ টেবিল	নেটওয়ার্ক, রাস্তা	গ্রাফ

পরীক্ষার আগের রাত

1. উচ্চতা h (রুট 0): সর্বোচ্চ $2^{h+1} - 1$, সর্বনিম্ন $h + 1$ নোড
2. N লেভেল: সর্বোচ্চ $2^N - 1$ নোড; full ট্রিতে লিফ = ইন্টারনাল + 1
3. অ্যারে ট্রি: সন্তান $2i$, $2i + 1$; প্যারেন্ট $\lfloor i/2 \rfloor$
4. Pre/post + inorder → একক ট্রি; pre + post শুধু full হলে
5. BST-র inorder সাজানো
6. দুই সন্তানসহ BST delete → inorder successor
7. B+ tree n: $\leq n - 1$ কী, লিফ ভাঙলে কী ওপরে কপি
8. তুলনা-সর্ট $\geq \Omega(n \log n)$ (decision tree)
9. Quick: গড়ে $n \log n$, সাজানো + প্রথম pivot-এ n^2
10. Merge: $T(n) = 2T(n/2) + n = n \log n$, $O(n)$ মেমোরি
11. Binary search: $T(n) = T(n/2) + c = \log n$
12. BFS → কিউ, unweighted সংক্ষিপ্ততম পথ; DFS → স্ট্যাক
13. Dijkstra গ্রিডি, ঋণাত্মক এজ নয়; Kruskal $O(E \log E)$
14. হ্যাশ chaining $O(1 + \alpha)$
15. DP = D&C + ওভারল্যাপিং উপ-সমস্যা সংরক্ষণ
16. Hanoi $2^n - 1$ চাল; Fibonacci DP $O(n)$

→ 49th BCS ICT 2025-এর DSA MCQ এক বাক্সে

প্রশ্ন	উত্তর	প্রশ্ন	উত্তর
BFS ব্যবহার করে	**কিউ**	যে ট্রাভার্সাল BST সাজায়	**inorder**
সর্টিংয়ের নিম্নসীমা প্রমাণ	**decision tree**	তুলনা-ভিত্তিক নয়	**counting sort**
insertion sort-এর worst তুলনা	**$n(n-1)/2$**	সবচেয়ে দ্রুত বৃদ্ধিহার	**$O(\log n)$**
হ্যাশ chaining, load α	**$O(1+\alpha)$**	suffix tree-র কাজ	**স্ট্রিং সার্চ**

শীর্ষ ফাঁদ BST নয় এমন ক্রম থেকে "BST বানাও" → তা বলুন · bubble sort অসাজানো ডেটায় চলে; binary search চলে না · activity selection সাজায় **end** সময়ে · ছাপা BST হিপ নয় · randomized quicksort-এর worst case তবুও n^2 · DFS optimal নয় · লিংকড লিস্টে binary search $O(n)$

গড় খরচ	Access	Search	Insert	Delete
অ্যারে (অসাজানো)	1	n	n	n
লিংকড লিস্ট	n	n	1 (নোড জানা)	1 (নোড জানা)
স্ট্যাক / কিউ	top / front 1	n	1	1
হ্যাশ টেবিল	–	1	1	1
BST (ব্যালান্সড / হেলানো)	–	$\log n$ / n	$\log n$ / n	$\log n$ / n
বাইনারি হিপ	max 1	n	$\log n$	$\log n$ (max)

মনে রাখার কৌশল জটিলতার সিঁড়ি "$1 \cdot \log \cdot \sqrt{} \cdot n \cdot n \log n \cdot n^2 \cdot 2^n \cdot n!$": প্রশ্নের বৃদ্ধিহার পড়ুন, ধাপটি খুঁজুন, উত্তর দিন।

✓ অনুশীলন MCQ (আগে যেগুলো এসেছে)

আগে নিজে করুন, পরে মিলান

১. BFS-এ ব্যবহৃত ডেটা স্ট্রাকচার: ✎ 49th BCS ICT 2025

ক) স্ট্যাক খ) কিউ
গ) হিপ ঘ) লিংকড লিস্ট

২. যে ট্রাভার্সাল BST-কে সাজানো ক্রমে দেয়: ✎ 49th BCS ICT 2025

ক) preorder খ) postorder
গ) level order ঘ) inorder

৩. তুলনা-ভিত্তিক নয় এমন সর্ট: ✎ 49th BCS ICT 2025

ক) counting sort খ) merge sort
গ) heap sort ঘ) quick sort

৪. Insertion sort-এর worst-case তুলনা: ✎ 49th BCS ICT 2025

ক) $n(n+1)$ খ) $n \log n$
গ) $n(n-1)/2$ ঘ) $n(n-1)/4$

৫. হ্যাশ টেবিল, chaining, load factor α: গড় সার্চ ✎ 49th BCS ICT 2025

ক) $O(\alpha^2)$ খ) $O(\log \alpha)$
গ) $O(\alpha)$ ঘ) $O(1+\alpha)$

৬. Randomized quicksort-এর worst case: ✎ Combined Bank Officer(IT) 2019

ক) $O(n^2)$ খ) $O(n \log n)$
গ) $O(n)$ ঘ) $O(\log n)$

৭. N লেভেলের বাইনারি ট্রিতে সর্বোচ্চ নোড: ✎ Combined Bank AP 2020

ক) 2^N খ) $2^N - 1$
গ) N^2 ঘ) $2N$

৮. Push 200, 500, 100, তারপর Pop দেয়: ✎ BAPEX AGM(ICT) 2023

ক) 200 খ) 500
গ) 100 ঘ) কিছুই না

৯. Huffman coding-এর time complexity: ✎ Combined Bank Officer(IT) 2019

ক) $O(n)$ খ) $O(n^2)$
গ) $O(\log n)$ ঘ) $O(n \log n)$

১০. Binary search-এর জটিলতা: ✎ Combined Bank Officer(IT) 2018

ক) $O(\log n)$ খ) $O(n)$
গ) $O(n \log n)$ ঘ) $O(1)$

উত্তরমালা → ১-খ · ২-ঘ · ৩-ক · ৪-গ · ৫-ঘ · ৬-ক · ৭-খ · ৮-গ · ৯-ঘ · ১০-ক

লিখিত প্রশ্ন pre[] = {1, 2, 4, 8, 9, 5, 3, 6, 7}, post[] = {8, 9, 4, 5, 2, 6, 7, 3, 1}: বাইনারি ট্রিটি আঁকুন এবং এর inorder লিখুন। ১০ নম্বর

✎ **যে পরীক্ষায় এসেছে:** BPDB AE(CSE) 2024 · BUET M.Sc. April 2016

১. রুট 1 (pre-এর প্রথম, post-এর শেষ); pre[1] = 2 বাম সন্তান ২
২. post-এ 8 9 4 5 2 (2 পর্যন্ত) বাম সাবট্রি, 6 7 3 ডান ৩
৩. পুনরাবৃত্তি: 2 → (4 → 8, 9), 5; 3 → 6, 7; আঁকুন (পৃ. ৩) ৩
৪. Inorder **8 4 9 2 5 1 6 3 7**; pre + post শুধু full ট্রিতে একক ২

→ পরীক্ষার আগে ✓

✓ দুটি ট্রাভার্সাল থেকে ট্রি পুনর্গঠন
✓ জটিলতার টেবিল মুখস্থ
✓ BFS/DFS + Dijkstra ট্রেস
✓ অ্যারে বনাম লিংকড লিস্ট, স্ট্যাক বনাম কিউ